Kuku Localise

Series-aware dubbing localisation for microdramas

1. The problem, and why I picked it

The localisation editor who adapts a 50-episode show into Hindi (or another language) for dubbing. The problem: episodes are translated one at a time, so the show stops sounding like one show.

Three failures follow:

- **Identity drift.** The same person appears under different cues.
- **Register drift.** English “you” must become aap / tum / tu in Hindi, and the choice carries the power dynamic. It also depends on gender, which English cues never state.
- **Object and term drift.** One heirloom is called “amethyst pendant”, “oval pendant”, “locket” and “medallion” across three episodes; each would be translated differently if episodes run independently.

2. Assumptions

- Output is a dubbing script (voice actor reads it), so each line must fit its time slot, not just be accurate.
- Target language is Hindi for the prototype; the code takes any language name, but the speech-rate budget (~18 characters/sec) and register rules are calibrated for Hindi only.
- A human native-speaker reviewer signs off; the tool produces a draft and flags where it is unsure.
- Story Bible fields (gender, relationships) are inferred from screenplay text by an LLM, so they can be wrong and are reviewed by a human before use.

3. How the solution works (6-step pipeline)

Principle: the LLM does creative work; Python enforces anything measurable.

Step	Who does it	What it does
1. Parse	Code	Scenes, timestamps, clean speaker cues (strips nested parentheticals and O.S./V.O.), per-beat duration in seconds.
2. Story Bible	LLM + human review	Per character: gender, address register, relationships, spoken name; glossary of recurring terms. Generic cues (OFF-SCREEN VOICE, STRANGER) or zero-overlap merges are flagged for mandatory review, never merged silently.
3. Translate	LLM	Prompt carries the character profile, previous 2–3 beats, scene context and the beat’s time budget.
4. Validate	Code	Speech rate vs ~18 chars/sec; register check by Unicode-aware regex against the bible; subtitle fit (max 44 chars/line, 2 lines).
5. Repair	LLM	Failing beats are re-sent with an exact directive (e.g. “too long by 28%, shorten to <= 36 chars”). Max 2 retries, then tagged Honest Failure for a human.
6. Culture pass	LLM	Finds US idioms (“raincheck”, “ballpark”) and attaches a localised equivalent with rationale.

4. How it scales to every show

- Bible once per show, reused for every episode and every target language.
- Content-hash cache (SHA-256 of provider, model, system, user): reruns and UI refreshes cost zero tokens.
- --no-baseline skips the episode-alone comparison run, which exists only to prove the idea; production needs just the series run, cutting calls and tokens by 50%.
- Cost tracking in usage.json (calls, tokens, estimated USD per episode). Episodes can run in parallel, limited only by API rate limits.
- Where it gets expensive: repair retries on long Hindi lines, and human review time. Code checks cut review to flagged beats.

Success in a real team: a native reviewer's edit rate (characters changed / characters delivered) and review minutes per episode should fall versus the episode-alone draft, with no rise in post-release complaints about names or register.

Known failures and limits:

- On tightly linked early episodes (1–3) the baseline already sees most context, so the series-aware gain is small. The gain should appear in later episodes where a relationship has shifted.
- The 18 chars/sec budget is a rule of thumb, not measured on real dubbing sessions; code cannot judge naturalness.
- Bible gender and register are LLM inferences; a wrong entry propagates to every episode unless the reviewer catches it (hence mandatory review of low-confidence merges).
- The culture pass can over-adapt; its output is a suggestion with rationale, not an automatic change.

5. Tools used

Tools: Gemini API for translation, Streamlit for UI.